PRE-PAPER · WORKING DRAFT

Agentic Weight-Space Programming as an Alternative to Fine-Tuning in Small Language Models

Authors: Luiz Melo · **Affiliation:** Universidade do Sul de Santa Catarina (Unisul) · **Draft:** November 9, 2026

Working draft – status of each result and remaining work are listed at the end.

TL;DR

- **The defect.** Small "thinking" models often get stuck repeating themselves: *"Wait, let me check… Yes… Wait…"*. Out of every 100 grade-school math problems, Qwen3.5-0.8B runs out of space on 85 before writing an answer.
- **The fix.** We gave Claude agents the model's weights and a small testing budget, and asked them to fix this by editing the weights directly, with no training. **With four tiny edits, the model went from solving 15% to 67% of our practice problems, and from 8% to 47% on harder competition math it never saw during the search.**
- **The comparison.** The edit beats the one published in *Sparks of Weight-Space Programming*, beats banning the word "Wait", and beats fine-tuning. Fine-tuning reached 29%, using 13× more GPU time. The edited model also writes 60% shorter answers, so it is cheaper to run.
- **What mattered.** Two independent agents found the same fix. A script with the same tools and budget, but no agent reasoning, did not. Every cheap shortcut score we gave the agents was eventually gamed, and only real test runs kept them honest. Surprisingly, giving agents more interpretability data made their fixes worse.
- **Where it stops working.** On a second, diffuse defect (following formatting instructions, IFEval), neither the agents, nor a system prompt, nor fine-tuning improved the model. Every candidate that looked better in practice checks lost 3–7 points on fresh prompts. The method works when the defect is concentrated and its effect is large; it cannot find effects smaller than the test's own sampling noise.

Everything ran on one gaming graphics card (GTX 1660 SUPER, 6 GB).

A small model that loops is usually treated as a training problem: collect examples of good reasoning and fine-tune. That works, but for a looping model even collecting the examples is expensive, because most of its attempts never end.

Recently, Takehi et al. showed something different. A frontier model (GPT-6 "Astra") read the raw weights of this same model and edited 0.016% of them, and MATH-500 accuracy went from 7.9% to 48.1%. We wanted to know three things:

1. **Does it reproduce,** on a cheap setup, and what exactly did that edit do?
2. **Is it more than a trick?** Could a decoding setting or ordinary fine-tuning do the same?
3. **What in the agent's process actually matters?**

The defect: a model that can't stop thinking

Qwen3.5-0.8B "thinks" in a scratchpad before answering and ends its thinking with a special `</think>` marker. With a budget of 12,288 tokens (about 9,000 words), most of its answers never get there.

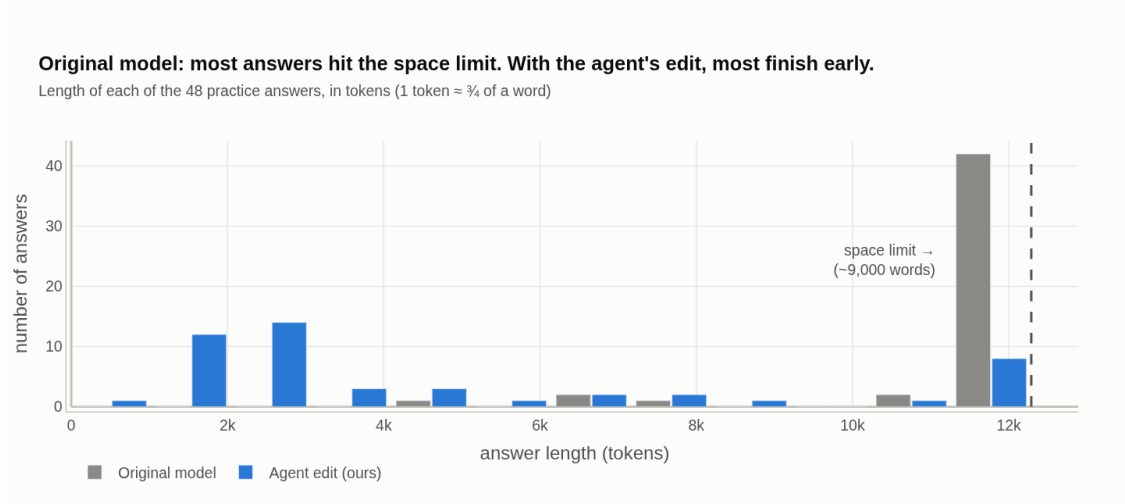


Figure 1: Length of each of 48 practice answers. Most of the original model's answers hit the space limit; with the agent's edit, most finish early.

Prompting does not help. A system prompt telling the model not to repeat itself does no better than a meaningless placebo prompt. Both actually *raise* the loop rate slightly (31% and 30%, against 23% without a prompt). The loop lives in the model's habits, not in its instructions.

The rules of the game

Setup

- **Model:** Qwen3.5-0.8B, the same revision as the original study.
- **Task:** grade-school math (GSM8K) with thinking on and a 12,288-token budget.
- **Edits allowed:** at most 4. Each edit picks a group of "neurons" (columns of an output matrix in one layer), a direction in the model's internal space, and a strength. It then changes only how strongly those neurons write along that direction:

$$w_j \leftarrow w_j - \alpha\,(u^\top w_j)\,u, \qquad |\alpha| \le 10.$$

  In words: $\alpha = 1$ deletes that part of what the neurons write, $\alpha > 1$ flips it, and $\alpha < 0$ makes it stronger. Each edit is a rank-1 change.

- **Agents get:** the weights, a written brief, and a scorer with a fixed budget. The scorer offers 40 "quick scores", which measure how much the model *wants* to repeat itself without letting it write. It also offers 6 "test runs", in which the edited model really solves 16 practice problems.
- **Not allowed:** training, running the model outside the scorer, editing the vocabulary layers, or reading the original study's edit, our notes, other agents' work or the scorer's code. These rules were not enforced in software: we audited every action in every agent's log afterwards and found no violations.

- **Agents:** Claude Opus 5.5, one per run, about 2 hours each. Some runs also got an **atlas**: a detailed scan of the model's internal activity while looping.
- **How we judge:**
  - **Practice set:** we confirm each agent's pick on 48 other problems (dev48).
  - **Locked test:** we run one pre-registered pick per method, once, on 32 sealed test problems.
  - **Harder math:** we check transfer to 148 competition problems (MATH-500) that no search ever touched.
- **Fair comparisons:** every method answers the same problems with the same random seeds, so we can count, problem by problem, who solved what.

Can an agent fix the loops by editing weights?

EXPERIMENT 1

Can we reproduce the published edit?

The published edit changes only two matrices, in layers 15 and 19. We recovered it exactly. Each change is a single rank-1 edit with strength 6: it takes the part of 64 neurons' output that lies along one "repetition direction" and flips it to $-5\times$ its original size. Both layers use almost the same direction, written by different neurons.

On our setup, the published edit lifts the practice set from 15% to 46%, and harder math from 8% to 21%. Our absolute MATH numbers are lower than the paper's because our token budget is 12k rather than 32k. All methods here share our budget.

EXPERIMENT 2

Can a Claude agent find its own fix?

Our first agent (a0) only had quick scores. It found a different edit, one that discourages the model from copying "Yes", "?" and "Wait" in two attention layers, and it matched the published edit on the locked test (44% vs 41%).

Then we added the 6 test runs. **Two agents, working independently, converged on the same fix, and it is much stronger: 67% and 65% on the practice set.**

agent	what the edit does ($\leq$ 4 rank-1 edits)	practice set
a1	weakens the urge to say "Wait" in 3 neurons (L17n1915, L18n881, L22n573) and strengthens the `</think>` signal in the last layer	**67%** (26 rescued, 1 lost)
a2	removes "Wait" from 3 neurons (two of them the same as a1's) and strengthens `</think>` in layer 22	**65%** (24 rescued, 0 lost)

How does it do it: both agents read the weights "through the vocabulary". For each neuron, they asked which word it pushes the model to say. That is how they found neurons that push "Wait". Their notes show they learned from their failures. Negating those neurons outright made the model say "wait wait wait…" and solve nothing, which told them the neurons work in the opposite way from what they had first assumed.

The two parts only work together. Our ablations, each a test run on 16 problems:

- the `</think>` boost alone solves 19%;
- the "Wait" suppression alone solves 44–50%;
- both together solve 69%, and almost nothing runs out of space.

A loop needs a way back in ("Wait, let me check") and no way out. The edit closes the first and opens the second.

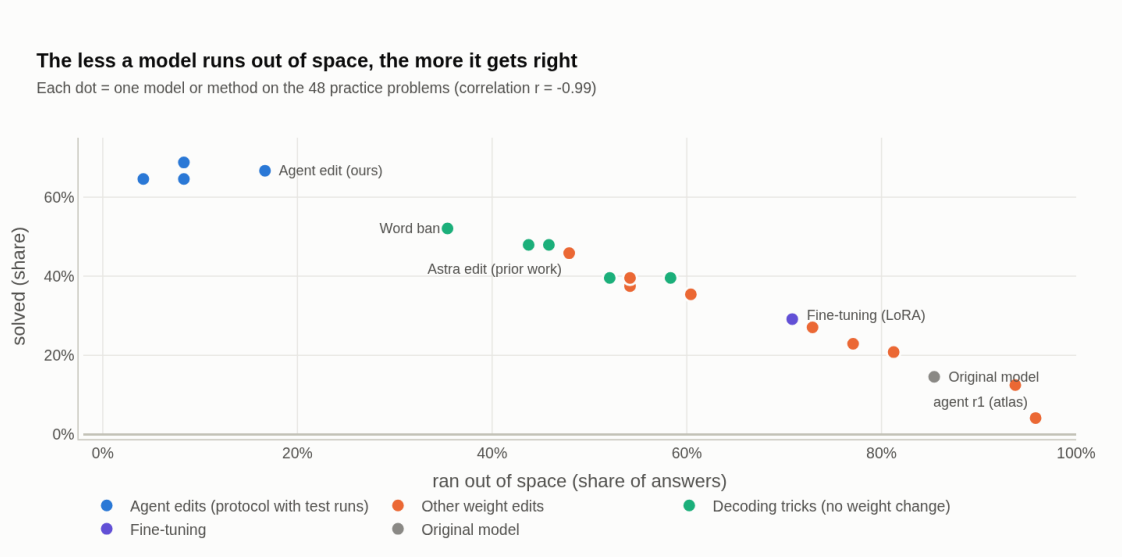


Figure 2: Across all 20 models and methods we tested, the fewer answers that run out of space, the more are correct ($r = -0.99$).

EXPERIMENT 3

Does it hold on problems it never saw?

On the locked test, the agent's edit solves 56% (the original: 13%; the published edit: 41%). On harder competition math, which was never used in any search, it solves 47%, against 21% for the published edit. Compared problem by problem with the published edit on MATH, it rescued 44 problems and lost 6.

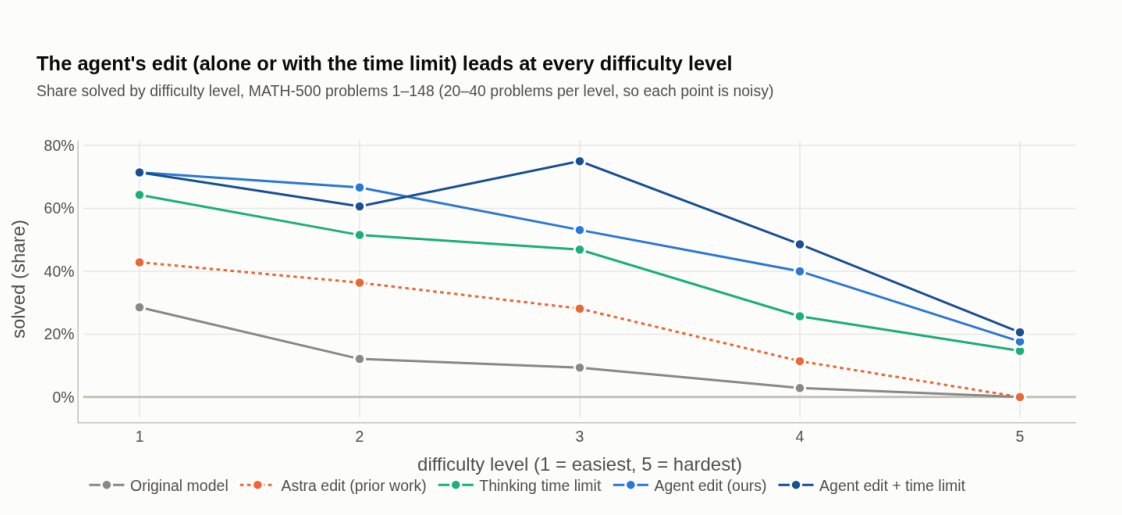


Figure 3: The agent's edit (alone or with a thinking time limit) stays ahead at every difficulty level.

With the edit, thinking pays off again: at 67%, the edited model beats the original model with thinking switched off (about 48%).

A second, larger locked test confirms it. We opened 200 more locked problems once, after writing down our predictions. The agent's edit solves 61.5% (original 15.5%, published edit 53.5%). Problem by problem, it beats the published edit by **8.0 ± 3.4 points** (32 rescues, 16 losses); over all 232 locked problems, by 9.1 ± 3.1.

Is it just a trick?

A skeptic would ask whether the edit is a fancy way of doing something simpler. We tried the simpler things, tuned on 48 problems (three times what the agents saw), which favors them.

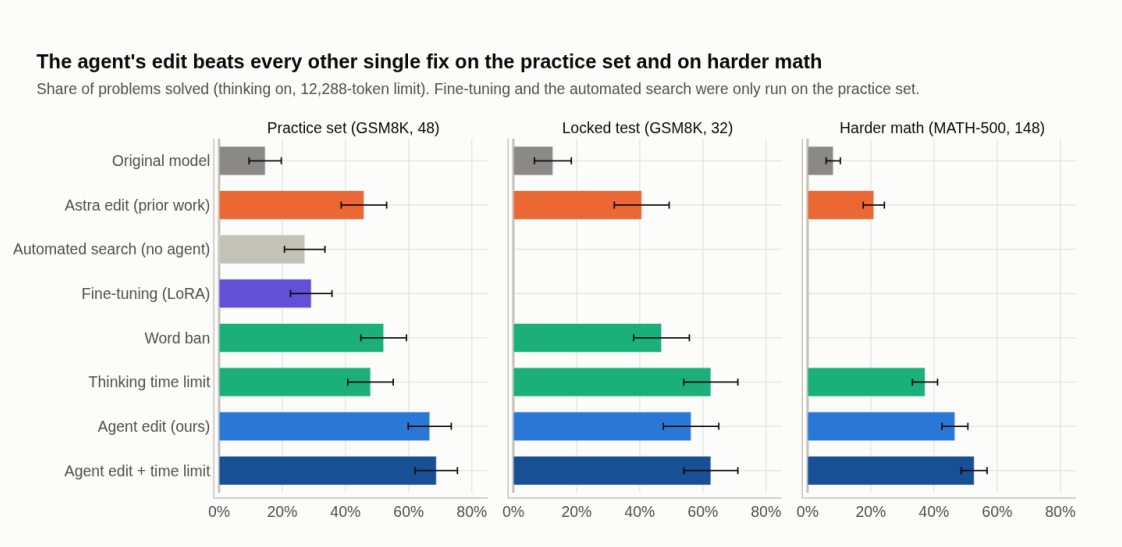


Figure 4: Share of problems solved by each method on the three problem sets (error bars: ±1 standard error).

method	practice set (48)	locked test (32)	harder math (148)
original model	15%	13%	8%
published edit (Astra)	46%	41%	21%
ban the word “Wait” (best of 3 settings)	52%	47%	27% (first 48)
thinking time limit, 4,096 tokens (best of 2)	48%	**63%**	37%
automated search, no agent	27%	—	—
fine-tuning (LoRA)	29%	—	—
agent’s edit (a1)	**67%**	56%	**47%**
agent’s edit + time limit	**69%**	**63%**	**53%**

EXPERIMENT 4

Could a decoding setting do the same?

Setup

- **Word ban:** while the model writes, make “Wait” (and variants) nearly impossible and nudge `</think>`. No weight change.
- **Time limit (“budget forcing”):** after N tokens of thinking, cut it off and force an answer. No weight change.

Banning the word is worse everywhere. The edit is not a global ban. It works through neurons that fire *when the model is looping*, so it leaves “Wait” alone when it is useful.

The time limit is a strong baseline on grade-school math and ties the edit on the locked tests (61.0% vs 61.5% on the 200-problem test; +0.5 ± 3.4). But it gets there at a higher cost: twice as many answers hit the token cap (34% vs 17%), and it writes about 40% more tokens. Banning “Wait” stops the model from restarting, but it does not make it conclude. On longer problems it hurts, because it also cuts off reasoning that was going somewhere. On harder math the edit

wins by 9.5 points, rescuing 23 problems and losing 9. **Together they do best of all (53% on harder math, 69.5% on the 200-problem locked test, 8 points above either alone, with the fewest cut-off answers)**: the edit breaks loops from the inside, and the time limit catches what is left.

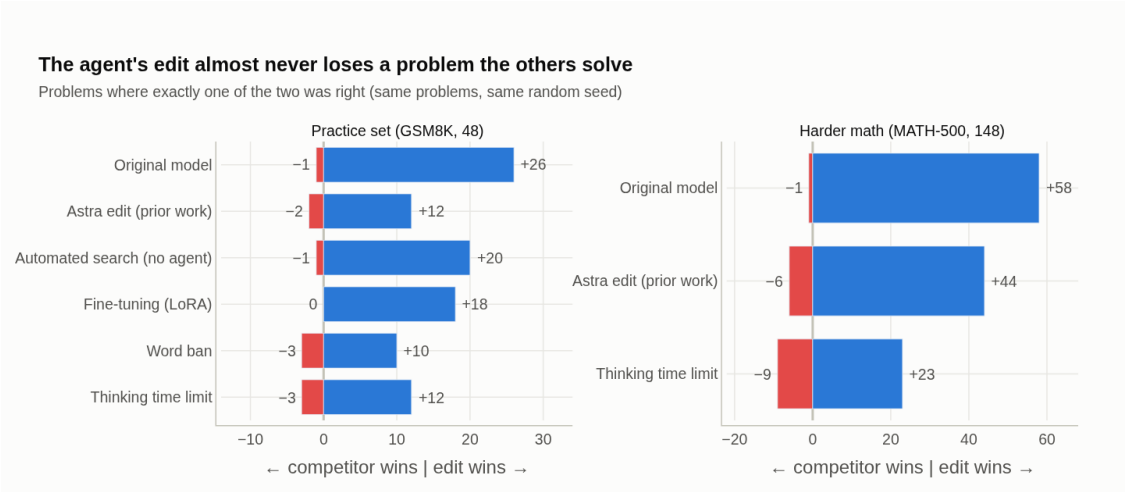


Figure 5: Problem by problem, same problems and seeds: problems only the agent's edit solved (right) against problems only the other method solved (left).

EXPERIMENT 5

Could ordinary fine-tuning do the same?

Setup

- **Data:** the model's own reasoning on 600 *other* GSM8K problems. Only 57 attempts finished cleanly and correctly. For 279 that looped, we cut the reasoning where the loop began, closed the thinking and let the model answer: 118 answers were correct. That gives 175 training examples.
- **Training:** LoRA, a standard and cheap fine-tuning method (rank 16, all layers, 3 rounds). We picked the best round on the same 16 problems the agents used for test runs.

Fine-tuning reached 29% on the practice set. That is better than the original (15%), but problem by problem the agent's edit rescued 18 problems that fine-tuning missed and lost 0. The fine-tuned model still ran out of space on 71% of its answers (the edit: 17%). It learned what to write *after* a cut, not *when* to stop. This is one standard recipe. More data or reinforcement learning might do better, but each costs more GPU time (see below).

EXPERIMENT 6

Is the agent needed, or just its tools?

We wrote a script that tries edits from the same toolbox the winning agents used: "word directions" for the 300 most common words, the neurons most aligned with them, random strengths. It had the same budget, 40 quick scores plus 6 test runs on the most promising candidates.

The script's best test run solved 3 of 16, and its pick reached 27% on the practice set, 40 points below the agents. The tools and the feedback are not enough. What matters is what the agent chooses to try.

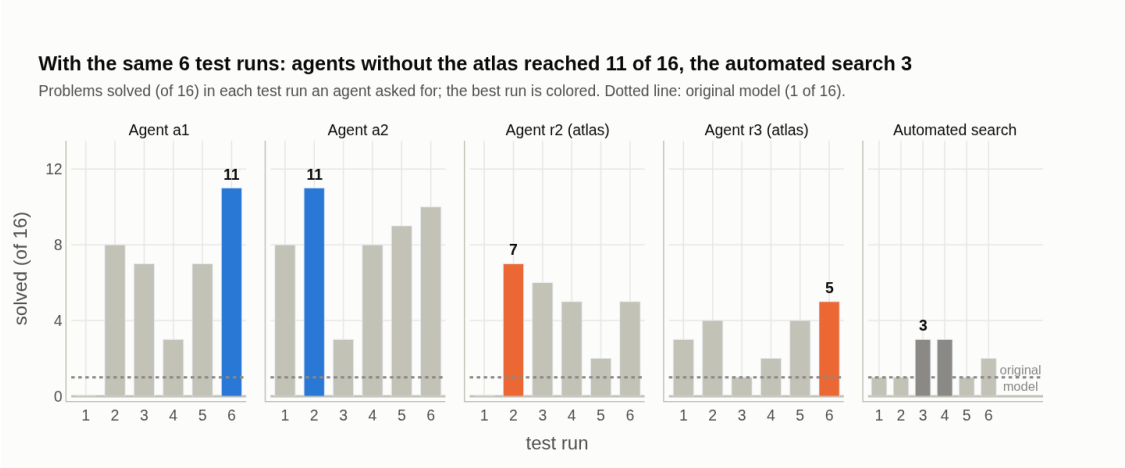


Figure 6: Each searcher's 6 test runs (16 problems each). The two agents without the atlas reached 11 of 16; the agents with the atlas 5–7; the script 3.

What we learned about the process

EXPERIMENT 7

Can a cheap score stand in for real tests?

Real test runs are slow (16 problems take about 10 minutes on our GPU), so the obvious move is a fast proxy. Every proxy we tried broke once a strong optimizer pushed on it:

- an edit that scored better than the published edit on the model's "surprise" at loop words gained 6 points in real use, against 28 for the published edit;
- an edit with no word-for-word loops at all still ran out of space 77% of the time;
- **agent r1 drove the quick score to 20× the published edit's value, and its edit solved 4% of problems, worse than doing nothing.** It blocked word-for-word copying, and the model simply looped in other ways (alternating values, growing lists).

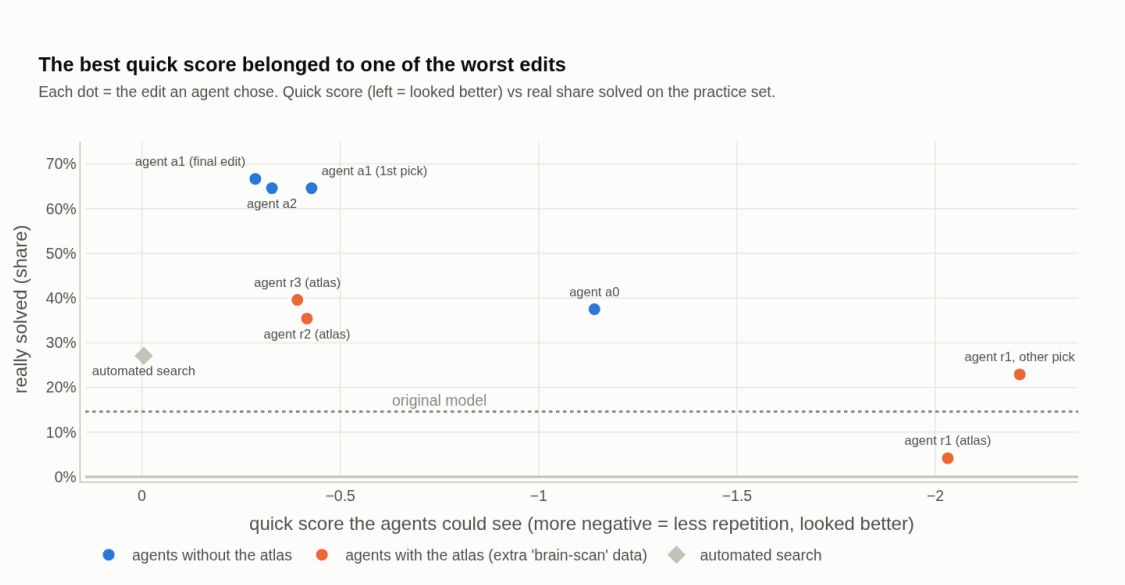


Figure 7: The quick score each agent's chosen edit achieved (left = looked better) against how often it actually solved problems.

Six real test runs fixed this. Under that protocol, both agents' best-scoring early ideas solved 0 of 16 in a test run, and the agents dropped them. **Our takeaway: an agent's objective must**

come from real behavior, even a small sample of it. Proxies are useful as guard rails, not as goals.

EXPERIMENT 8

Does more interpretability data help?

We ran three pairs of agents with identical instructions; in each pair, one agent also had the atlas.

In all three pairs, the agent without the atlas found the better edit (38% vs 4%; 67% vs 35%; 65% vs 40%). With the atlas, agents chased intricate internal mechanisms, such as a "re-checking" state in the middle layers and "loop heads". These looked convincing in the data but did not stop the loops. Without it, agents went for the direct levers: the word "Wait" and the `</think>` signal. Three pairs make this suggestive, not proven.

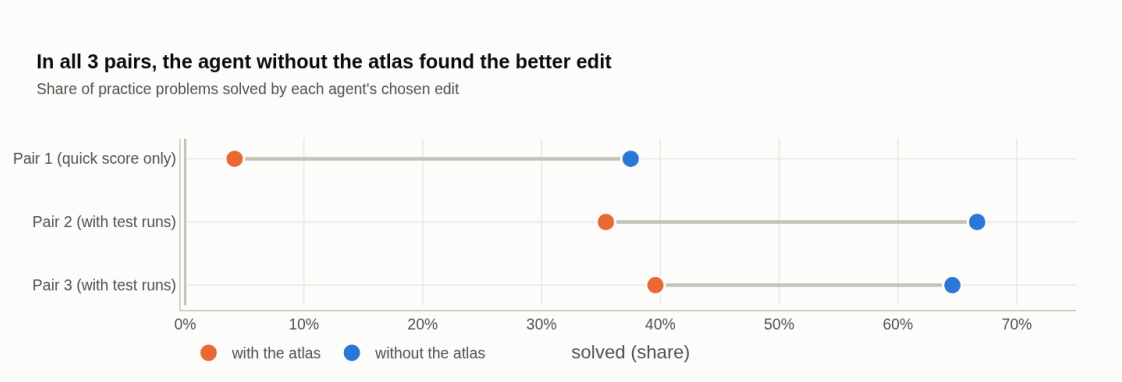


Figure 8: Each pair of agents had identical instructions; the agent without the atlas did better every time.

What did it cost?

	practice set	GPU-hours (1660 SUPER)	agent usage
agent's edit (one run)	**67%**	**1.3**	5.4 M tokens read, 48 k written (≈ $5–7 at API prices)
automated search	27%	1.6	—
tuning the decoding tricks	48–52%	4.4	—
fine-tuning (LoRA)	29%	17.1	—

- **One-time cost.** One agent run took about 2 hours, which is roughly the electricity of a microwave running for 15 minutes. Fine-tuning took 17 GPU-hours, 10 of them just to generate training data from a model that wastes most of its attempts.
- **Running cost.** The edited model's answers are **60% shorter** on the practice set and 29% shorter on harder math. This saving repeats on every question it answers.
- **Compared with the original agent.** The published run used 86.8 M tokens read and 2.9 M written over 20 hours (from its public logs). Our agent used 16× and 60× less, in about 2 hours. The tools differ, so this is not a controlled comparison.
- **The fine print.** We ran on a subscription, so the dollar figures are list-price estimates. On a very cheap rented GPU, fine-tuning could cost a few dollars less than the agent's usage, for less than half the improvement.

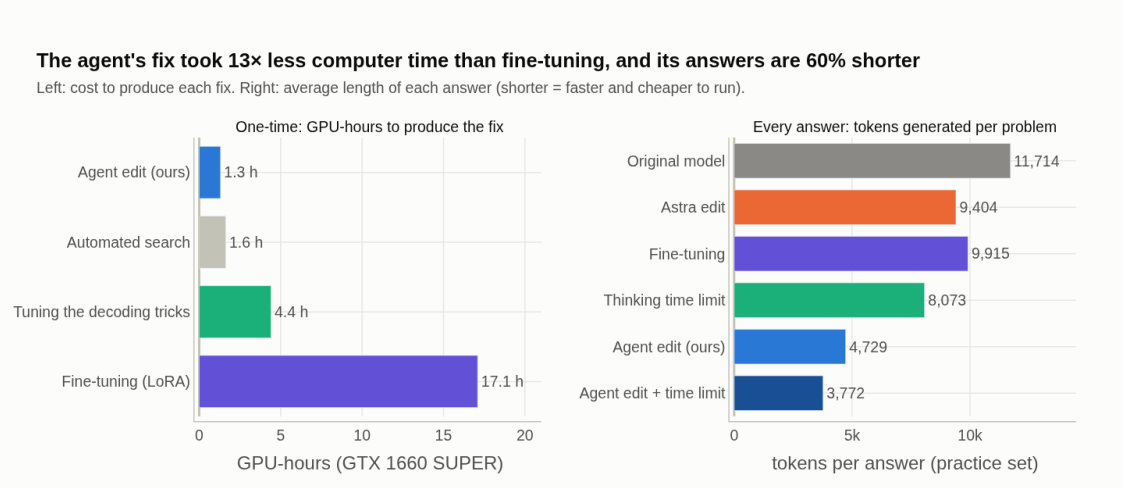


Figure 9: Left: GPU-hours to produce each fix. Right: average answer length, which sets the cost of every future answer.

Did the surgery break anything?

With thinking switched off, none of the edits changes math accuracy (200 problems) or tool-calling accuracy (300 requests) beyond about two standard errors. The edit acts where the loops happen, during thinking, and leaves other skills alone. One side effect is positive: with thinking on, the published edit gets better at declining irrelevant tool calls (16 of 16 vs 11 of 16), probably because shorter thinking leaves less room to hallucinate a call.

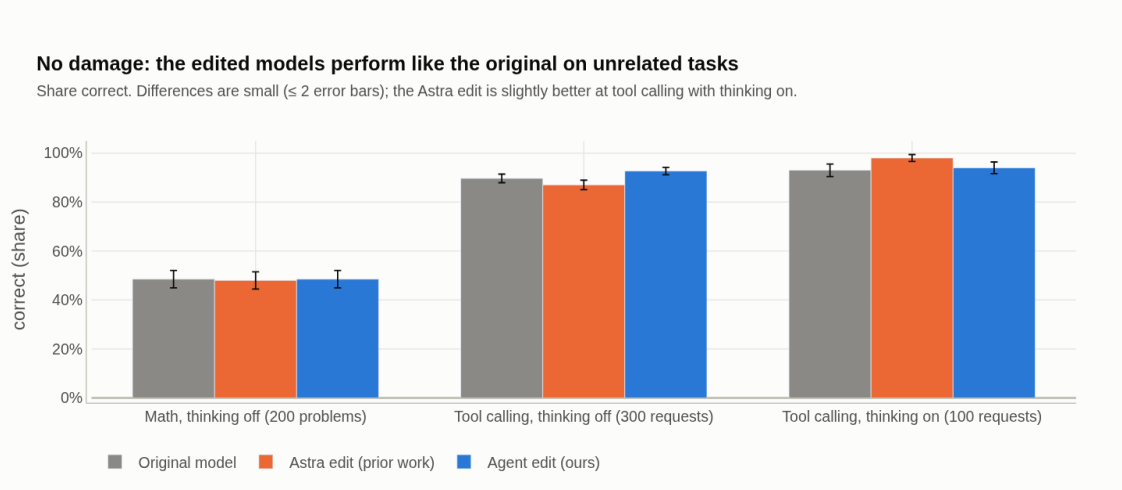


Figure 10: Accuracy on unrelated tasks for the original model, the published edit and the agent's edit.

Does it work on a second defect?

No, and the way it fails is informative. We needed a defect with no obvious keyword. Broken JSON and tool calls turned out not to be a problem for this model with thinking off: it was near ceiling. So we chose **instruction following** (IFEval: "no commas", "all lowercase", "end with this exact phrase", "at least 3 highlighted sections"). With thinking off, the model follows every instruction in only about 55% of prompts. Its failures are spread over many instruction types.

Setup

- **Same protocol as for the loops:** an agent with 40 cheap probe scores and 6 real generation checks, at most 4 rank-1 edits.

- **Comparisons:** a system prompt asking for care, and LoRA fine-tuning on 414 of the model's own answers that passed the official checker (same recipe as before).
- **Confirmation:** on 152–200 IFEval prompts never used in any search.

arm	change vs original on fresh prompts
agent, 48-prompt checks (b1)	-3.3 ± 4.1
the loop edit (a1), not tuned for this	-3.9 ± 4.3
system prompt	-6.6 ± 4.2
fine-tuning (epoch chosen on 48 prompts)	-7.2 ± 4.3
agent, 128-prompt checks (b2)	-6.0 ± 3.7 (200 prompts)

Every method looked better in its practice check (+5 to +12 points) and lost on fresh prompts. The reason is sampling noise. Two runs of the *unchanged* model with different random seeds disagree on 23% of prompts. On 48 prompts, that gives a margin of error of 8–9 points, larger than any plausible gain. A larger, 128-prompt check did not rescue the agent either: the best edit's own opposite-sign control also looked slightly positive.

What this means. On the loops, the effect was +46 points and 16 problems were enough to see it. On instruction following, cheap changes of any kind (weights, prompt, gradient) seem to move the model away from a point that is already close to the best nearby, and no check of this size can find the rare changes that help. **The method's reach is set by the signal-to-noise ratio of the real test, not only by the agent.** Fine-tuning hits the same wall, so this is not a case of the agent losing to gradients.

The agent of the second run also found, from the weights alone, a direction in the residual stream that 99.7% of the decay-gate rows of all 288 DeltaNet heads read with the same sign: a global "forgetting" control channel. Editing it changes the model, but not its instruction following. We leave it for future work.

Background and related work

Weight-space programming. Takehi et al. (2026) showed that frontier models can read and write neural network weights directly, including the loop-fixing edit we reproduce. We use their edit as our reference and their public logs for the cost comparison.

Model editing. Rank-one edits have been used to change facts stored in a model (ROME, Meng et al., 2022; MEMIT, Meng et al., 2023). Deleting a single direction can remove a behavior such as refusal (Arditi et al., 2024). Our edits belong to this family. The difference is that an agent chooses them by reasoning about behavior, instead of a fixed formula.

Steering. Adding a vector to the model's internal state at run time can steer its behavior (ActAdd, Turner et al., 2023; representation engineering, Zou et al., 2023). Our edits are permanent instead of applied at run time.

Overthinking. Reasoning models often re-check more than they need to (Chen et al., 2024 [verify]). Known fixes include suppressing words like "Wait" (Wang et al., 2025 [verify]) and capping thinking time (Muennighoff et al., 2025). We use both as baselines.

Agents and Goodhart. Agents already run interpretability experiments (MAIA, Shaham et al., 2024). Strong optimizers are known to game proxy objectives (Gao et al., 2023; Skalse et al., 2022). We see the same with proxies derived from interpretability.

What can't we claim yet, and what comes next?

- **Sample sizes:** the main comparisons now rest on a 200-problem locked test ($\pm$3–4 points), but dev and MATH numbers still rest on 48–148 problems.
- **One model, one defect that works:** "Wait" is an easy word to target, and the second defect (IFEval) gave a well-controlled negative result. **Next:** a concentrated defect without a keyword, and a second model. Any second defect needs a real test whose noise is well below the expected effect, measured before the search.
- **Few replicates:** the winning protocol has 2 agents, and the atlas comparison has 3 pairs. **Next:** two more of each.
- **One fine-tuning recipe:** a stronger recipe could close part of the gap, at a higher cost.
- **Honor system:** the agents' restrictions were audited, not enforced.
- **Different sampler from the original study:** absolute numbers shift, but every comparison here uses the same settings.

The claim, as it stands: for concentrated behavioral defects with a large effect, an agent editing weights can be a faster, cheaper and more interpretable alternative to fine-tuning. For diffuse defects, such as instruction following in a 0.8B model, neither it nor cheap fine-tuning helped.

Data, code and logs

Everything is in the project repository:

- **Lab notebook:** dated, with predictions written before results (`docs/CADERNO.md`).
- **Records of every agent run:** briefs, notes, scorer logs, and every edit submitted (267).
- **Figures:** the notebook that draws every chart here (`notebooks/results.ipynb`).
- **Cost:** a detailed study (`docs/cost.md`).

The agents' full transcripts and the raw generations (~210 MB) will be released after review.

Status of this draft and plan to submission

	status
Reproduction and dissection of the published edit	done
6 agent runs (2 protocols, 3 with/without-atlas pairs)	done
Decoding baselines, naive search, fine-tuning baseline	done
Transfer to harder math, side effects, cost	done
Locked test with $\geq$ 200 problems	done (t200: 5 methods; the word-ban and LoRA arms still to run)
More replicates (2 + 2 agents)	to do
Second defect without a keyword (IFEval)	done: negative, with fine-tuning and prompt baselines
Concentrated second defect; second model	to do (needed for a main-track venue)
Re-grade MATH with a symbolic checker	to do
Verify citations; edit-anatomy diagram	to do

Venue: arXiv preprint first, then an interpretability or LLM-agents workshop with the current results, and a main-track conference if a concentrated second defect works.

Alternative titles: *Agents as Weight-Space Programmers: Fixing Reasoning Loops in Small Language Models*; *Generation in the Loop: What Makes Agentic Weight-Space Programming Work*.

References (to verify and complete)

- Takehi, R., Teoh, J., Zhang, M., Khattab, O. (2026). *Sparks of Weight-Space Programming*. MIT CSAIL. https://rikiyat.github.io/blog/weight-space-programming/. Logs: https://github.com/RikiyaT/lm_understand-logs.
- Meng, K., Bau, D., Andonian, A., Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. NeurIPS.
- Meng, K., Sen Sharma, A., Andonian, A., Belinkov, Y., Bau, D. (2023). Mass-Editing Memory in a Transformer. ICLR.
- Arditi, A., et al. (2024). Refusal in Language Models Is Mediated by a Single Direction. NeurIPS.
- Turner, A. M., et al. (2023). Steering Language Models With Activation Engineering. arXiv.
- Zou, A., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv.
- Muennighoff, N., et al. (2025). s1: Simple Test-Time Scaling. arXiv.
- Chen, X., et al. (2024). Do NOT Think That Much for 2+3=? On the Overthinking of o1-like LLMs. arXiv. [verify]
- Wang, C., et al. (2025). Wait, We Don't Need to "Wait"! Removing Thinking Tokens Improves Reasoning Efficiency. arXiv. [verify]
- Shaham, T. R., et al. (2024). A Multimodal Automated Interpretability Agent. ICML.
- Gao, L., Schulman, J., Hilton, J. (2023). Scaling Laws for Reward Model Overoptimization. ICML.
- Skalse, J., et al. (2022). Defining and Characterizing Reward Hacking. NeurIPS.
- Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.
- Cobbe, K., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv.
- Hendrycks, D., et al. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS.
- Lightman, H., et al. (2023). Let's Verify Step by Step. arXiv.